

UNIVERSITY OF GHANA
DEPARTMENT OF COMPUTER SCIENCE
CSCD602 — ADVANCED SOFTWARE ENGINEERING

TECHNICAL DEBT PLAN

ICCTECH: A Web-Based IT Service Management and Helpdesk System

Student Name	Clement Asamoah
Student ID	22424193
Project	ICCTECH
Academic Year	First Semester, 2025/2026
Examination Duration	48 Hours
Live Application	http://45.79.223.146:8080/index.php
Report Version	1.0 — Final

Prepared for the Individual Software Engineering Capstone Examination

Document Control

Document	Technical_Debt_Plan.pdf
Purpose	Identification, prioritisation, management and repayment plan for significant ICCTECH technical debt.
Primary Basis	Defined 48-hour project scope, deployed Linode/Docker architecture and submitted source-code configuration.
Debt Items	9 significant technical-debt items.
Highest Priority	Credential management and HTTPS hardening.
Review Status	Prepared for final examination submission; release traceability must be verified during final packaging.

Executive Summary

This Technical Debt Plan documents significant technical debt identified during the ICCTECH 48-hour software-engineering project. Technical debt is treated as a managed engineering trade-off rather than simply as unfinished functionality. The plan identifies the cause and impact of each debt item, assigns a priority and classification, and defines a practical resolution path.

Nine significant debt items are recorded. Two are classified as critical security-hardening concerns requiring immediate attention before long-term production use, several high-priority items are scheduled for the first maintenance release, and medium-priority architecture and maintainability items are accepted temporarily or scheduled for later evolution. The plan also explains how debt will be reviewed, repaid and prevented from accumulating without visibility.

The presence of technical debt does not imply that the core examination workflow failed testing. Functional acceptance and technical debt address different questions: testing verifies whether the defined requirements behave as expected, while technical-debt management identifies design, security, operational and maintainability improvements that remain desirable after the timeboxed implementation.

Table of Contents

Document Control	2
Executive Summary	3
1. Purpose and Context	6
1.1 Purpose	6
1.2 Definition of Technical Debt	6
1.3 Project Context	6
2. Technical Debt Management Approach	7
2.1 Identification Sources	7
2.2 Required Debt Record Structure	7
2.3 Priority and Classification	8
3. Technical Debt Register	9
4. Detailed Debt Analysis	11
4.1 TD-01 — Credentials in Docker configuration	11
4.2 TD-02 — HTTP rather than HTTPS	12
4.3 TD-03 — Published MySQL host port	13
4.4 TD-04 — Limited automated regression and CI	13
4.5 TD-05 — No off-host backup and restore	14
4.6 TD-06 — Limited monitoring and alerting	14
4.7 TD-07 — Single-host application and database	15
4.8 TD-08 — Legacy/upstream naming	15
4.9 TD-09 — Documentation and evidence drift	16
5. Repayment Plan and Roadmap	17
5.1 Immediate Security and Submission Actions	17
5.2 First Maintenance Release	17
5.3 Future Evolution	17
5.4 Repayment Sequence	17
6. Review, Governance and Change Control	19
6.1 Proposed Review Procedure	19
6.2 Definition of Debt Repaid	19
7. Relationship to Testing and Acceptance	20
8. Residual Risk and Project Limitations	21
9. Conclusion	22

10. References and Acknowledgements 23

List of Tables

Table 1. Technical debt classification model	8
Table 2. ICCTECH technical debt register	9
Table 3. Technical debt repayment roadmap	17

List of Figures

Figure 1. Technical debt identification and recording process	7
Figure 2. Technical debt classification model	8
Figure 3. Current ICCTECH deployment architecture and associated debt	10
Figure 4. Target production-hardened architecture after debt repayment	12
Figure 5. Technical debt repayment roadmap	18
Figure 6. Review, governance and change-control cycle	19
Figure 7. Functional testing versus technical-debt management	20

1. Purpose and Context

1.1 Purpose

The purpose of this plan is to provide a formal record of technical debt identified in ICCTECH and to show how that debt will be prioritised, monitored and repaid after the 48-hour examination period. It complements the SRS, Testing Report, Deployment section and Maintenance Strategy by documenting engineering compromises that are acceptable temporarily but should not remain invisible.

1.2 Definition of Technical Debt

For this project, technical debt is any implementation, architecture, security, testing, deployment or maintainability compromise that reduces short-term effort but creates future cost, risk or rework. Technical debt may be deliberate, such as accepting a single-server architecture for a short examination, or incidental, such as retaining legacy internal naming because broad refactoring would create unnecessary regression risk.

1.3 Project Context

ICCTECH was completed under a strict 48-hour capstone examination. The project therefore prioritised a demonstrable IT Service Management workflow, requirements traceability, testing, deployment and documentation. This time constraint made it necessary to defer some production-hardening, automation and operational-resilience work to later releases.

2. Technical Debt Management Approach

2.1 Identification Sources

Significant technical debt was identified from the following complementary reviews of the delivered system:

- Source-code and configuration review, including Docker and database configuration.
- Deployment architecture review of the Linode-hosted environment.
- Testing and requirements traceability review.
- Security-hardening review of authentication, transport and network exposure.
- Maintainability review of internal naming, automation and release traceability.
- Operational review of backup, monitoring, resilience and future scaling needs.

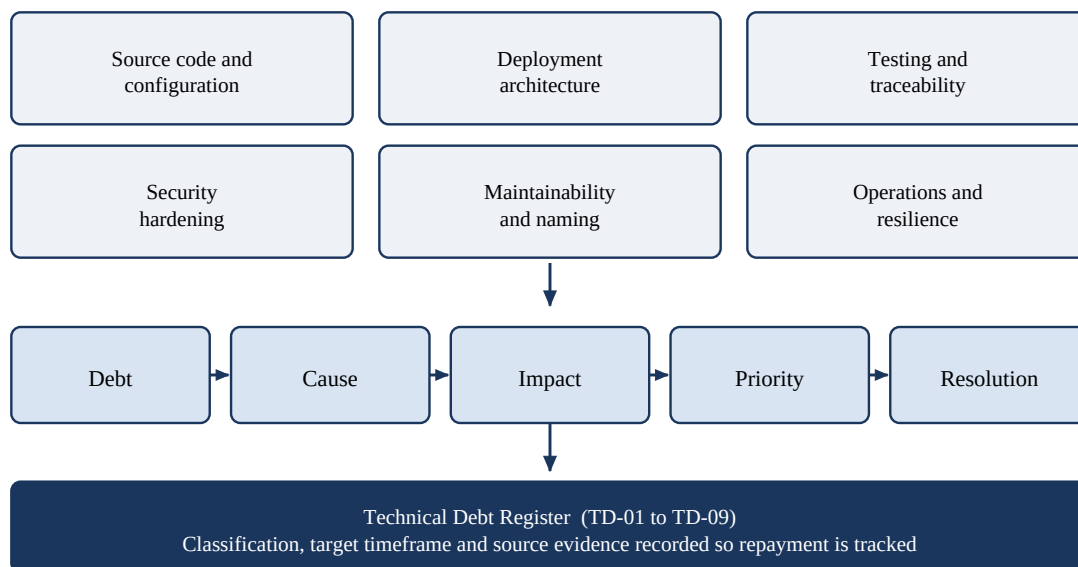


Figure 1. Technical debt identification and recording process

2.2 Required Debt Record Structure

Each significant technical-debt item is recorded using the examination structure: Debt → Cause → Impact → Priority → Proposed Resolution. The plan additionally records a classification, target timeframe and source evidence so that repayment can be tracked rather than treated as an informal intention.

2.3 Priority and Classification

Table 1. Technical debt classification model

Classification	Meaning	Treatment
Critical / Immediate	Creates unacceptable security or release risk for continued production use.	Resolve before long-term public/production operation.
High / Scheduled	Does not prevent examination demonstration but materially affects security, reliability or regression confidence.	Schedule into production hardening or first maintenance release.
Medium / Managed	Acceptable temporarily under the examination scope but affects maintainability, resilience or future scalability.	Document, review each release and repay according to operational need.

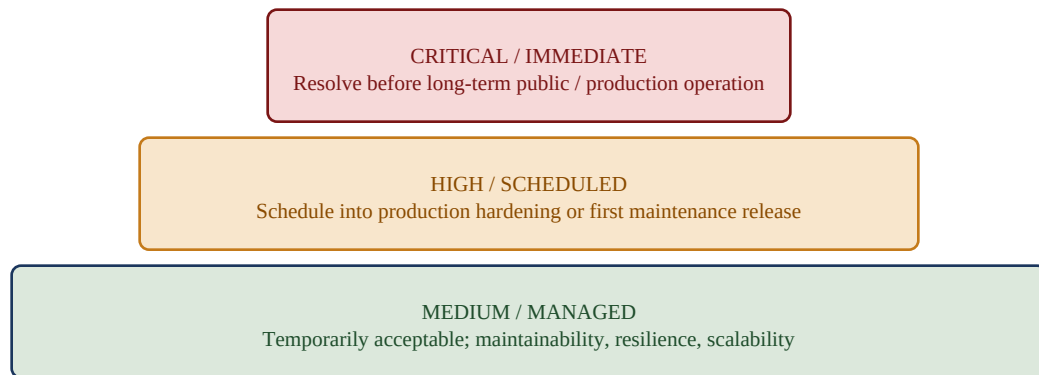


Figure 2. Technical debt classification model

3. Technical Debt Register

Table 2 summarises the significant debt items identified for the current ICCTECH release. The detailed analysis in Section 4 expands each item using the required debt-management structure.

Table 2. ICCTECH technical debt register

ID	Technical Debt	Priority	Classification	Target	Status
TD-01	Development/default credentials remain in Docker configuration	Critical	Immediate resolution	Before continued public/production use	Outstanding / managed
TD-02	Live deployment currently uses HTTP rather than HTTPS	Critical	Immediate resolution	Before long-term production use	Outstanding / managed
TD-03	MySQL host port is published in the Docker Compose deployment	High	Production hardening	Before hardened production use	Outstanding / managed
TD-04	Project-specific automated regression coverage and CI are limited	High	Scheduled for next release	First maintenance release	Outstanding / managed
TD-05	No demonstrated automated off-host backup and restore process	High	Scheduled for next release	First maintenance release	Outstanding / managed
TD-06	Production monitoring and automated alerting are limited	Medium	Scheduled	Subsequent maintenance release	Outstanding / managed
TD-07	Application and database share a single Linode host	Medium	Temporarily acceptable / future evolution	Future production evolution	Outstanding / managed
TD-08	Legacy/upstream naming remains in internal configuration and resources	Medium	Temporarily acceptable / scheduled refactoring	Future refactoring releases	Outstanding / managed
TD-09	Documentation and evidence can drift from the exact deployed release	Medium	Resolve before submission	Before final Sakai submission	Outstanding / managed

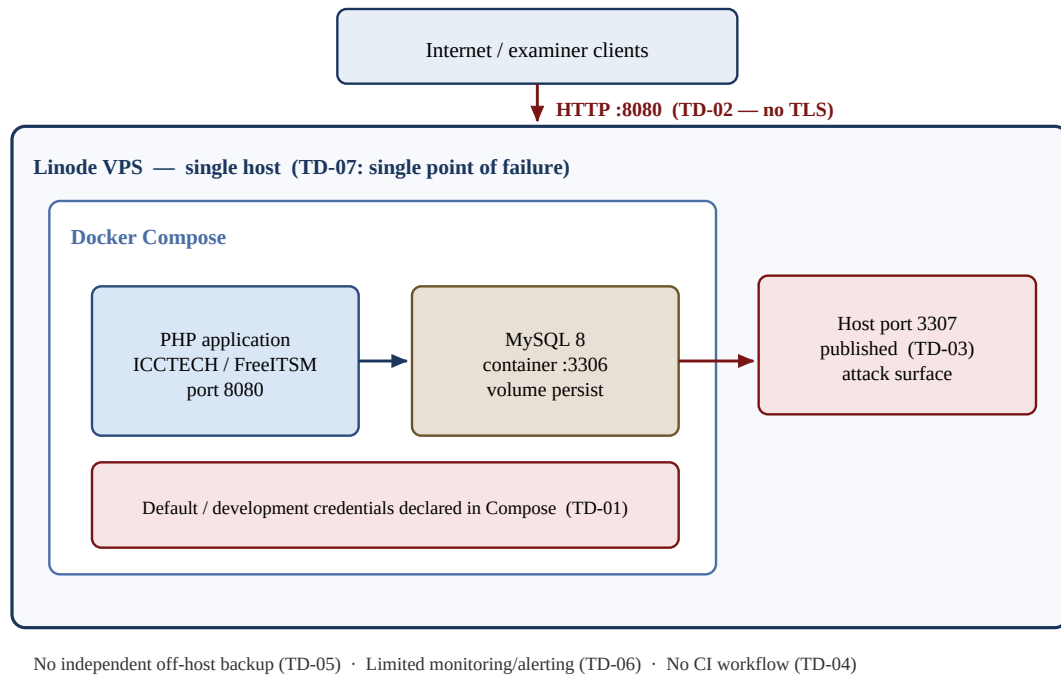


Figure 3. Current ICCTECH deployment architecture and associated debt (TD-01, TD-02, TD-03, TD-07; TD-04/05/06 noted)

Figure 3 locates the register items on the deployed architecture. TD-01, TD-02, TD-03 and TD-07 appear as structural call-outs on the single Linode host; TD-04, TD-05 and TD-06 are operational gaps noted beneath the host boundary. Section 4 records each item in the required Debt, Cause, Impact, Priority and Proposed Resolution structure.

4. Detailed Debt Analysis

Each item below follows the required record structure. Priority and classification are repeated from the register so that the analysis can be read independently of Table 2.

4.1 TD-01 — Development/default credentials remain in Docker configuration

Debt	Development/default credentials remain in Docker configuration.
Cause	The timeboxed Docker environment used directly declared development/default database credentials for rapid setup and reproducibility.
Impact	If development credentials are reused in an internet-accessible environment, exposure of the repository or configuration could increase the risk of unauthorised database access.
Priority	Critical
Classification	Immediate resolution
Proposed Resolution	Create unique production credentials, rotate any reused credentials, move secrets to protected environment variables or Docker secrets, and keep secret files outside source control.
Target	Before continued public/production use
Source Evidence	docker-compose.yml declares database credential variables directly.

Security note: the plan intentionally does not reproduce actual credential values. The correct action is to rotate and externalise production secrets rather than copy them into documentation.

4.2 TD-02 — Live deployment currently uses HTTP rather than HTTPS

Debt	Live deployment currently uses HTTP rather than HTTPS.
Cause	The application was deployed rapidly to the Linode server by IP address and port 8080 without TLS termination during the 48-hour project period.
Impact	Authentication and application traffic are not encrypted in transit, making the deployment unsuitable for long-term production use on untrusted networks.
Priority	Critical
Classification	Immediate resolution
Proposed Resolution	Assign a domain name, configure a reverse proxy or web-server virtual host, install a trusted TLS certificate, and redirect all HTTP requests to HTTPS.
Target	Before long-term production use
Source Evidence	The verified live application URL uses <code>http://45.79.223.146:8080/index.php</code> .

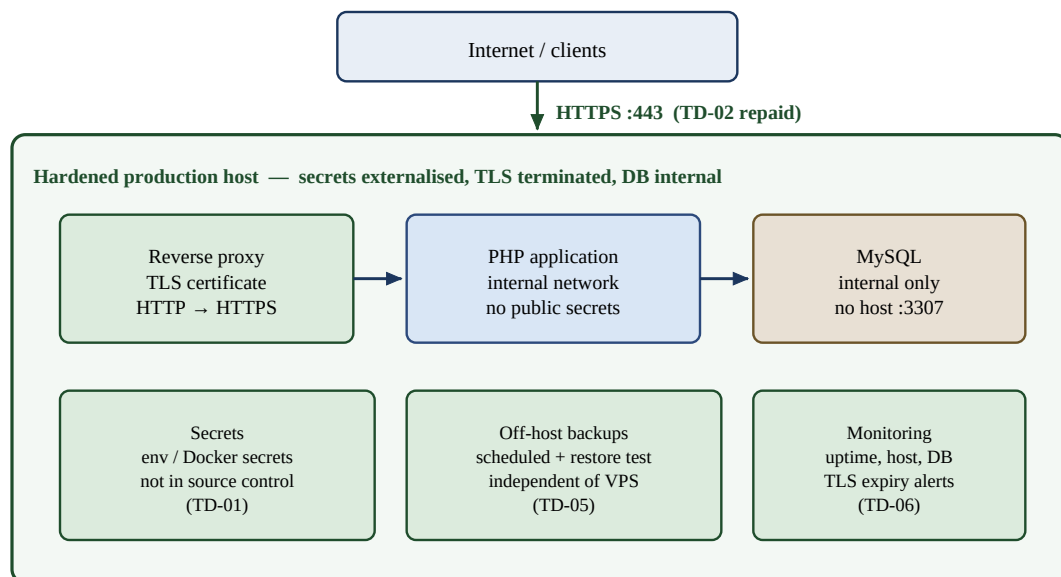


Figure 4. Target production-hardened architecture after repayment of TD-01, TD-02, TD-03, TD-05 and TD-06

4.3 TD-03 — MySQL host port is published in the Docker Compose deployment

Debt	MySQL host port is published in the Docker Compose deployment.
Cause	Publishing the database port simplified development, troubleshooting and direct database administration.
Impact	If host or cloud firewall controls allow remote access, unnecessary database exposure increases the attack surface.
Priority	High
Classification	Production hardening
Proposed Resolution	Remove the public database port mapping in the production Compose profile and use the internal Docker network. Where administrator access is needed, use SSH tunnelling or tightly restricted firewall rules.
Target	Before hardened production use
Source Evidence	docker-compose.yml maps host port 3307 to MySQL container port 3306.

4.4 TD-04 — Project-specific automated regression coverage and CI are limited

Debt	Project-specific automated regression coverage and CI are limited.
Cause	The 48-hour project prioritised completion and validation of the core ticket lifecycle. The repository contains automated test scripts, but there is no demonstrated CI workflow executing a requirements-mapped regression suite on every change.
Impact	Future modifications may introduce regressions that are discovered late or only during manual testing.
Priority	High
Classification	Scheduled for next release
Proposed Resolution	Expand automated tests for authentication, RBAC, ticket creation, assignment, status transitions, resolution and persistence; map tests to FR/NFR identifiers; add a GitHub Actions or equivalent CI pipeline.
Target	First maintenance release
Source Evidence	The repository contains tests/, while .github contains no workflow configuration.

4.5 TD-05 — No demonstrated automated off-host backup and restore process

Debt	No demonstrated automated off-host backup and restore process.
Cause	The examination focused on functional deployment and Docker volume persistence rather than disaster-recovery automation.
Impact	Container volumes protect against normal container recreation but do not by themselves protect against server loss, corruption or accidental destructive changes.
Priority	High
Classification	Scheduled for next release
Proposed Resolution	Implement scheduled MySQL backups, store copies independently of the Linode host, define retention rules, protect backup access, and perform documented restoration tests.
Target	First maintenance release
Source Evidence	Docker volumes provide persistence, but the submitted deployment does not demonstrate an independent backup/restore workflow.

Engineering distinction: persistent Docker volumes reduce data loss during normal container restart or recreation, but persistence is not equivalent to an independent backup and disaster-recovery capability.

4.6 TD-06 — Production monitoring and automated alerting are limited

Debt	Production monitoring and automated alerting are limited.
Cause	Implementation effort was concentrated on application functionality, testing and deployment rather than operational observability.
Impact	Application, database, container, disk or host failures may not be detected until a user reports a problem.
Priority	Medium
Classification	Scheduled
Proposed Resolution	Add uptime checks, container health monitoring, CPU/memory/disk metrics, database availability monitoring and threshold-based alerts. Include TLS certificate expiry monitoring once HTTPS is implemented.
Target	Subsequent maintenance release
Source Evidence	No project-specific production monitoring or alerting configuration is demonstrated in the submitted deployment assets.

4.7 TD-07 — Application and database share a single Linode host

Debt	Application and database share a single Linode host.
Cause	A single-server Docker deployment was the simplest, fastest and most economical architecture for the 48-hour examination.
Impact	The server is a single point of failure and limits high availability and horizontal scalability. A host outage affects both the web application and database.
Priority	Medium
Classification	Temporarily acceptable / future evolution
Proposed Resolution	For higher availability, separate application and database workloads, consider a managed or replicated database, implement health checks and backups, and introduce load balancing when scale justifies it.
Target	Future production evolution
Source Evidence	The current deployment runs the application and MySQL services through Docker Compose on one Linode server.

4.8 TD-08 — Legacy/upstream naming remains in internal configuration and resources

Debt	Legacy/upstream naming remains in internal configuration and resources.
Cause	Renaming every inherited internal identifier during the 48-hour period would create unnecessary regression risk and consume time better spent validating the core workflow.
Impact	Legacy names can confuse maintainers and reduce consistency between the ICCTECH project identity and internal configuration, database names, paths or documentation.
Priority	Medium
Classification	Temporarily acceptable / scheduled refactoring
Proposed Resolution	Refactor legacy identifiers incrementally, beginning with project-facing configuration and documentation. Run regression tests after each naming change and preserve required third-party attribution and licence notices.
Target	Future refactoring releases
Source Evidence	Examples of legacy FreeITSM identifiers remain in docker-compose.yml, database paths and application configuration.

Refactoring must preserve any required upstream attribution, copyright and licence notices. The objective is naming consistency and maintainability, not removal of legitimate third-party acknowledgement.

4.9 TD-09 — Documentation and evidence can drift from the exact deployed release

Debt	Documentation and evidence can drift from the exact deployed release.
Cause	Source code, documentation, screenshots and testing evidence were prepared rapidly during the same examination window.
Impact	If artifacts refer to different revisions, traceability is weakened and the examiner may see discrepancies between documentation, repository and live system.
Priority	Medium
Classification	Resolve before submission
Proposed Resolution	Freeze the final application version, commit and tag it, record the final Git commit hash, verify screenshots and documentation against that build, and avoid untracked changes after evidence capture.
Target	Before final Sakai submission
Source Evidence	Release traceability depends on final repository state and should be confirmed as part of submission packaging.

5. Repayment Plan and Roadmap

5.1 Immediate Security and Submission Actions

- TD-01: rotate/externalise production credentials and ensure secrets are not committed to the public repository.
- TD-02: introduce HTTPS before long-term production use.
- TD-03: remove or strictly restrict direct database host-port exposure.
- TD-09: freeze the final build, record the repository commit/tag, and verify all submitted evidence against that release.

5.2 First Maintenance Release

- TD-04: expand requirements-mapped automated regression tests and add CI execution.
- TD-05: implement automated off-host database backups and perform a documented restore test.
- TD-06: add baseline uptime, host, container and database monitoring with alerts.

5.3 Future Evolution

- TD-07: improve resilience when operational scale justifies separating or replicating application/database workloads.
- TD-08: refactor legacy internal naming incrementally while protecting upstream attribution and regression stability.

5.4 Repayment Sequence

Table 3. Technical debt repayment roadmap

Phase	Debt Items	Objective	Exit Condition
Immediate / pre-production	TD-01, TD-02, TD-03, TD-09	Reduce security and submission-traceability risk.	Secrets externalised; HTTPS plan/implementation complete for production; database exposure restricted; final release frozen and identified.
First maintenance release	TD-04, TD-05, TD-06	Improve regression confidence, recoverability and observability.	CI regression suite running; off-host backup + restore test documented; monitoring alerts operational.
Future evolution	TD-07, TD-08	Improve resilience, scalability and maintainability.	Architecture changes justified by scale; naming refactoring completed incrementally without breaking required attribution.

Figure 5 presents the same sequence as a visual roadmap. Each phase must meet its exit condition before the next phase is treated as complete.

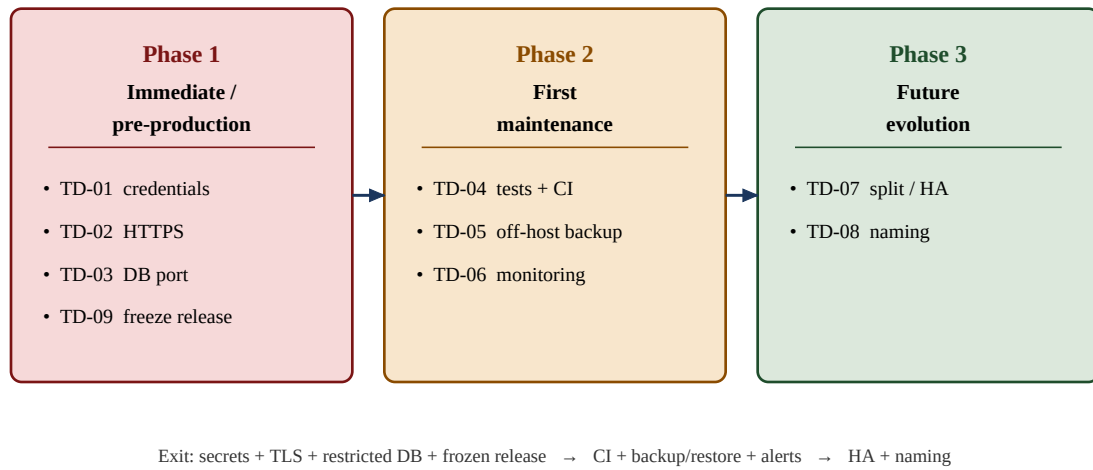


Figure 5. Technical debt repayment roadmap

6. Review, Governance and Change Control

The technical debt register should be reviewed at each maintenance release and whenever a significant architecture, security or deployment change is proposed. Critical security debt takes precedence over new features. High-priority reliability and test-automation debt is planned into the next release cycle. Medium-priority debt is ranked using user impact, operational risk, implementation effort and the likelihood that the debt will become more expensive to repay later.

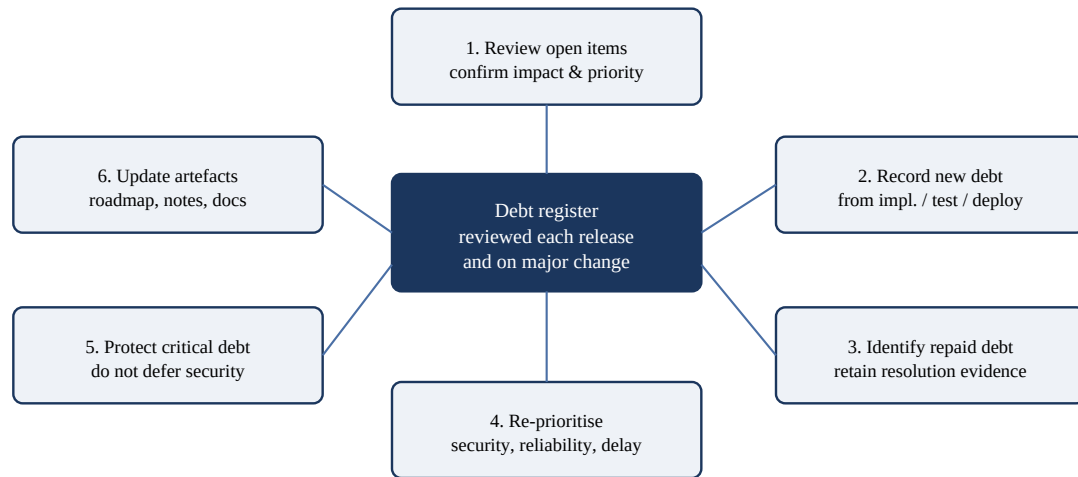


Figure 6. Review, governance and change-control cycle

6.1 Proposed Review Procedure

- Review open debt items and confirm whether the original impact and priority remain valid.
- Record newly introduced debt during implementation, testing or deployment.
- Identify debt that has been repaid and retain evidence of the resolution.
- Re-prioritise remaining items based on security, reliability, user impact and cost of delay.
- Ensure new feature work does not repeatedly defer critical security or data-protection debt.
- Update the roadmap, release notes and relevant project documentation.

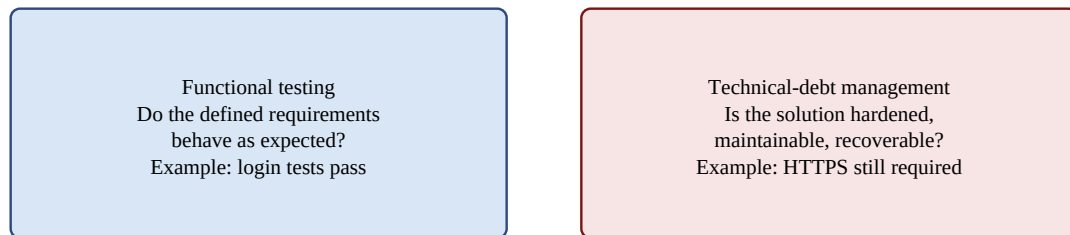
6.2 Definition of Debt Repaid

A debt item is considered repaid only when the proposed resolution has been implemented and verified. For example, TD-05 is not closed merely because a backup script exists; it should remain open until a backup is created, stored independently and successfully restored in a controlled test.

7. Relationship to Testing and Acceptance

The final ICCTECH testing cycle verifies whether the prioritised requirements behave as expected within the defined examination scope. Technical debt addresses a different dimension of software quality: whether the solution is sufficiently hardened, maintainable, observable, recoverable and scalable for future use. Therefore, a 100% pass rate for the selected functional test cases can coexist with open technical-debt items.

For example, a login function may pass all authentication tests while the deployment still requires HTTPS before long-term production use. Similarly, ticket persistence may pass service-restart testing even though an independent disaster-recovery backup process remains future work. The plan makes these distinctions explicit so that successful acceptance testing is not confused with complete elimination of engineering risk.



A 100% pass rate for selected functional tests can coexist with open debt items.

Ticket persistence may survive container restart while independent disaster-recovery backup remains future work (TD-05).

Acceptance testing $\neq$ elimination of engineering risk.

Figure 7. Functional testing versus technical-debt management

8. Residual Risk and Project Limitations

Until the critical and high-priority debt items are repaid, the current deployment should be regarded as suitable for examination demonstration and controlled evaluation rather than as a fully hardened enterprise production environment. The most important residual risks relate to transport security, secret management, database exposure, disaster recovery, operational monitoring and single-host availability.

The 48-hour project constraint also limits the amount of automated regression coverage and large-scale operational validation that can reasonably be completed. These limitations are intentionally documented rather than hidden, and they are connected to specific repayment actions in the roadmap.

9. Conclusion

ICCTECH demonstrates deliberate technical-debt management by identifying significant compromises, explaining why they exist, evaluating their impact, assigning priorities and defining practical repayment actions. The highest-priority debt concerns production security hardening, followed by regression automation, backup/recovery and monitoring. Architecture resilience and internal naming consistency are longer-term evolution items.

This approach ensures that the timeboxed examination solution remains understandable and maintainable after deployment. Technical debt is therefore treated as a controlled engineering obligation that is visible, prioritised and scheduled rather than as hidden unfinished work.

10. References and Acknowledgements

- University of Ghana, Department of Computer Science. CSCD602 Advanced Software Engineering Individual Project-Based Examination, First Semester 2025/2026.
- ICCTECH source repository and submitted project source code, including Docker deployment configuration and test assets.
- ICCTECH live deployment: <http://45.79.223.146:8080/index.php>.
- Docker documentation — containerisation, Compose networking, secrets and persistent storage concepts.
- MySQL 8.0 documentation — database security, backup and recovery concepts.
- PHP documentation — application runtime and security guidance.
- Linode/Akamai cloud hosting documentation — server networking, firewalls, backups and monitoring concepts.
- Upstream/third-party components used by the project are acknowledged in the main Project Documentation and repository licensing/attribution materials.